Universidad Mariano Gálvez
Sede Portales
Facultad de Ingeniería
Análisis de Sistemas 2

Proyecto Final: Análisis, Diseño Arquitectónico Moderno, Aplicación Funcional, UX/UI y Estrategia de Calidad

Informe final del proyecto CRM
Análisis del sistema actual, arquitectura, API, experiencia de usuario y estrategia de calidad

NOMBRE Y APELLIDOS	CARNÉ
Evelyn Luzmarie Martínez Hernández	9989-23-11816
Sergio Daniel Mayén Hernández	9989-23-12589
Carlos Emanuel Hernández García	9989-21-363

Fecha: 24 de octubre de 2026

Índice del informe

1. Introducción y resumen ejecutivo

2. Requerimientos, UML y gestión ágil

3. Evaluación arquitectónica, C4 y persistencia

4. Diseño de APIs, implementación funcional y BDD

5. UX/UI y Mobile-First

6. Estrategia de calidad y Plan SQA

7. Matriz de trazabilidad integral

8. Conclusiones y lecciones aprendidas

Anexos y evidencias pendientes

1. INTRODUCCIÓN Y RESUMEN EJECUTIVO

El proyecto analiza un CRM (Customer Relationship Management) orientado a centralizar la administración de clientes y contactos, el seguimiento de interacciones comerciales y tickets, la clasificación mediante categorías y etiquetas, la búsqueda avanzada y la importación/exportación de información.

Según el documento base, la solución utiliza Angular en el frontend, NestJS y TypeScript en el backend y Microsoft SQL Server como base de datos. La arquitectura actual es un monolito modular: API REST → Controller → Service → DatabaseService → SQL Server. Se contempla autenticación JWT y los roles SUP, ADM, OPR, CLI y SPT.

El propósito del informe es consolidar el análisis del sistema legado y organizar los componentes académicos solicitados: requisitos, modelado, evaluación arquitectónica, API, UX/UI y calidad. Se distingue entre capacidades descritas como actuales y mejoras que permanecen como propuestas.

Síntesis metodológica

La secuencia de trabajo comprende: (1) comprensión del sistema legado; (2) identificación de actores y procesos; (3) levantamiento de requisitos; (4) análisis de arquitectura y dependencias; (5) evaluación de persistencia y caché; (6) modelado UML/C4 y gestión ágil; (7) contrato API y BDD; (8) diseño centrado en usuario; (9) Mobile-First y accesibilidad; y (10) planificación SQA, pruebas y trazabilidad.

2. REQUERIMIENTOS, UML Y GESTIÓN ÁGIL

2.1 Especificación de requisitos (ISO/IEC/IEEE 29148:2018)

La especificación SRS separada identifica requisitos funcionales REQ-FN y no funcionales REQ-NF, prioridades y criterios iniciales de aceptación. Cubre autenticación JWT, usuarios, contactos, interacciones, tickets, catálogos, búsqueda, preferencias, perfiles sociales, alertas e importación/exportación. Debe validarse contra el código antes de aprobarse.

ID	Requisito resumido	Prioridad
REQ-FN-001	Validar credenciales de inicio de sesión.	Alta
REQ-FN-002	Emitir JWT tras autenticación válida.	Alta
REQ-FN-004	Crear, consultar, actualizar y buscar contactos.	Alta
REQ-FN-006	Registrar interacciones asociadas a contactos.	Alta
REQ-FN-009	Filtrar contactos mediante criterios soportados.	Alta
REQ-NF-001	No exponer secretos en respuestas o registros.	Alta
REQ-NF-004	Separar controladores, servicios y acceso a datos.	Alta
REQ-NF-008	Mantener OpenAPI alineado con la API real.	Alta

2.2 Diagramas UML 2.5.1

La entrega debe incluir fuentes editables y exportaciones de diagramas de clases, secuencia, actividades con calles de responsabilidad y paquetes. El documento base explica módulos y dependencias, pero no demuestra que todos los diagramas UML requeridos estén terminados; deben insertarse o enlazarse desde docs/uml/.

2.3 Fichas de diseño de componentes (SDD, IEEE 1016-2009)

Cada ficha debe describir propósito, responsabilidades, interfaces, entradas/salidas, dependencias, errores y pruebas. Como módulos de partida: Auth, Users, Contacts, Interactions, Catalogs, Tickets, Alerts, Search, Contact Preferences, Social Profiles e Import/Export. La ficha final debe cotejarse con el código real.

2.4 Gestión ágil

Se recomienda un tablero Scrum o Kanban. El Product Owner prioriza valor y valida criterios; el analista mantiene trazabilidad. Las historias deben documentarse con Card, Conversation y Confirmation, y revisarse con INVEST: independientes, negociables, valiosas, estimables, pequeñas y comprobables.

DoR: historia clara, valor definido, criterios verificables, dependencias identificadas y tamaño estimable. **DoD:** código integrado, revisión aprobada, pruebas ejecutadas, documentación actualizada, criterios cumplidos y defectos críticos resueltos. Añadir el enlace real al tablero.

3. EVALUACIÓN ARQUITECTÓNICA, C4 Y PERSISTENCIA

3.1 Estilos y escenarios de calidad (ISO/IEC 25010)

El monolito modular simplifica despliegue, pero concentra carga y puede aumentar acoplamiento. La arquitectura por capas refuerza responsabilidades; Repository abstrae persistencia; microservicios permiten despliegue y escalado por dominio con mayor complejidad operativa; EDA facilita procesos asíncronos pero exige reintentos, idempotencia y observabilidad. La comparación es cualitativa, no un benchmark ejecutado.

Estilo	Escalabilidad	Mantenibilidad	Observación
Monolito modular actual	Media	Media	Menor complejidad operativa; dependencia compartida de SQL Server.
Capas + Repository	Media	Alta	Mejora testabilidad y aislamiento de persistencia.
Microservicios (propuesta)	Alta por dominio	Alta si límites claros	Más coste de despliegue, seguridad y monitoreo.
EDA (complementaria)	Alta en tareas asíncronas	Media/alta	Exige consistencia eventual y manejo de reintentos.

3.2 Decisiones de arquitectura (ADR, ISO/IEC/IEEE 42010)

ADR-001 — Estilo arquitectónico: propuesta para mantener la modularidad, reforzar límites de dominio, contratos y acceso a datos; extraer microservicios solo ante necesidades medibles. Consecuencia: migración gradual, con disciplina de dependencias y pruebas.

ADR-002 — Persistencia políglota: propuesta no confirmada como implementada. SQL Server es la persistencia actual identificada. PostgreSQL podría cubrir dominios relacionales separados; MongoDB, documentos flexibles; Redis, caché temporal; Neo4j, relaciones complejas. No conviene incorporar los cuatro motores sin casos de uso y capacidad operativa definidos.

3.3 Modelo C4 (niveles 1 a 4)

Nivel 1: actores y CRM, con servicios externos solo si se configuran. **Nivel 2:** Angular, API NestJS y SQL Server; Redis y otros almacenes como objetivo propuesto. **Nivel 3:** Auth, Users, Contacts, Interactions, Catalogs, Tickets, Search, Alerts e Import/Export, organizados mediante Controllers, Services y DatabaseService. **Nivel 4:** clases críticas de controladores, servicios, DTO y acceso a datos. Adjuntar los diagramas exportados y sus fuentes desde docs/c4/.

3.4 Persistencia, Cache-Aside y consistencia

El acceso actual descrito es SQL Server mediante DatabaseService. Para Redis se propone Cache-Aside: consultar caché; si hay acierto, responder; si hay fallo, consultar SQL Server, almacenar el resultado con TTL y responder. Las mutaciones deben invalidar las claves de entidad y listas relacionadas.

Clave propuesta	TTL	Invalidación
crm:contacts:active:list	5 min	Crear/editar/activar/inactivar contacto
crm:contacts:{id}	15 min	Modificar contacto correspondiente
crm:catalogs:tags	60 min	Cambios en etiquetas
crm:catalogs:categories	60 min	Cambios en categorías
crm:contacts:search:{hash_filtros}	5 min	Cambios que afecten resultados

SQL Server debe conservar la autoridad de datos hasta que se apruebe otra estrategia. Con varios motores se deben definir propietario de datos, eventos/outbox, idempotencia, reintentos, compensación y reconciliación. La consistencia eventual es una propuesta, no una capacidad demostrada.

3.5 Patrones y cohesión/acoplamiento

Repository puede abstraer consultas SQL; Strategy, seleccionar estrategias de búsqueda/importación; Gateway, aislar integraciones externas; Observer, reaccionar a eventos; Circuit Breaker, proteger llamadas remotas inestables. El documento base no confirma que estos patrones estén implementados explícitamente; se describen como opciones de diseño.

4. DISEÑO DE APIS, IMPLEMENTACIÓN FUNCIONAL Y BDD

4.1 API-First y OpenAPI 3.0.3

La API REST conecta Angular y NestJS. OpenAPI debe documentar rutas reales, métodos, parámetros, esquemas, autenticación y respuestas. El documento base indica prefijo /api y servidor local http://localhost:3000/api. Confirmar las rutas exactas contra los controladores y la colección Postman.

4.2 Errores HTTP y Problem Details

El documento base describe respuestas success, message y data, y enumera códigos HTTP 200, 201, 400, 401, 403, 404, 409 y 500. RFC 9457 define application/problem+json; su adopción requiere adaptar y probar el manejador de excepciones, no solo documentarlo.

4.3 Catálogo de endpoints

Método	Endpoint documentado	Propósito	Estado
POST	/api/auth/login	Autenticación y JWT	Confirmar ruta/probar
GET	/api/contacts	Consultar contactos	Confirmar ruta/probar
POST	/api/contacts	Crear contacto	Confirmar ruta/probar
GET	/api/search/contacts	Búsqueda avanzada	Confirmar parámetros
POST	/api/interactions	Registrar interacción	Probar
GET	/api/tickets	Consultar tickets	Confirmar módulo/estado
GET	/api/catalogs/tags	Consultar etiquetas	Probar

4.4 BDD y aceptación (ISO/IEC/IEEE 29119-3)

Ejemplo ilustrativo de Gherkin; ajustar al contrato real:

```
Feature: Autenticación CRM
Scenario: Inicio de sesión válido
Given que existe un usuario activo con credenciales válidas
When se envía POST /api/auth/login con username y password
Then la API responde con éxito y un token JWT

Scenario Outline: Credenciales inválidas
Given que no se presentan credenciales válidas
When se envía POST /api/auth/login
Then la API rechaza la autenticación
Examples: | username | password |
```

Los archivos .feature, colección Postman y resultados automatizados deben adjuntarse como evidencias. No se afirma que el ejemplo haya sido ejecutado.

5. EXPERIENCIA DE USUARIO (UX/UI) Y MOBILE-FIRST

5.1 Diseño centrado en usuario

El recorrido principal contempla autenticación, consulta del panel, localización/creación de contactos, revisión del perfil, registro de interacciones y seguimiento de tickets. Los Customer Journey Maps deben documentar objetivos, pasos, fricciones y oportunidades por perfil.

5.2 Heurísticas y WCAG 2.2

Evaluar visibilidad del estado, correspondencia con el mundo real, control del usuario, consistencia, prevención/recuperación de errores y ayuda. WCAG 2.2 organiza la accesibilidad en POUR: perceptible, operable, comprensible y robusto. Revisar teclado, foco, etiquetas, contraste, errores y adaptación móvil. No declarar nivel AA sin auditoría documentada.

5.3 Mobile-First y Atomic Design

Diseñar primero para móvil y escalar a escritorio. Átomos: colores, tipografía, iconos, campos y botones; moléculas: búsqueda, campo etiquetado y tarjeta; organismos: formulario, listado, navegación y panel. Añadir el enlace compartible del prototipo Figma: [URL pendiente].

6. ESTRATEGIA DE CALIDAD Y PLAN SQA

6.1 Plan SQA (IEEE 730-2014) y Shift-Left

QA define procesos para prevenir defectos; QC inspecciona y prueba entregables. Shift-Left incorpora revisión de requisitos, análisis estático, validación DTO, pruebas unitarias y seguridad desde etapas tempranas. El plan debe definir roles, herramientas, auditorías, entregables y criterios de salida.

6.2 Pirámide de pruebas

Unitarias: servicios y validaciones aisladas. Integración: controladores, servicios y SQL Server con datos de prueba. E2E: login, CRUD de contactos, búsqueda e interacción. Rendimiento y seguridad complementan la pirámide. Adjuntar resultados; este informe no presume ejecución.

6.3 Gestión de defectos (IEEE 1044-2009)

Registrar identificador, versión, módulo, pasos, resultado esperado/actual, evidencia, severidad, prioridad, responsable y estado. Flujo recomendado: Nuevo → Clasificado → Asignado → En corrección → En verificación → Cerrado; reabrir si persiste. Severidad indica impacto y prioridad la urgencia.

7. MATRIZ DE TRAZABILIDAD INTEGRAL

Requisito	HU	Componente	Endpoint	Prueba/evidencia	Estado
REQ-FN-001/002	HU-01	Auth	POST /api/auth/login	autenticacion.feature; CP-01	Validar
REQ-FN-004	HU-02	Contacts	POST/GET /api/contacts	contactos.feature; CP-02	Validar
REQ-FN-009	HU-03	Search	GET /api/search/contacts	busqueda.feature; CP-03	Validar
REQ-FN-006	HU-04	Interactions	POST /api/interactions	interacciones.feature; CP-04	Validar
REQ-FN-007	HU-05	Tickets	GET /api/tickets	tickets.feature; CP-05	Validar
REQ-NF-001/002	HU-06	Auth/Guards	Endpoints protegidos	Pruebas 401/403; CP-06	Pendiente
REQ-NF-008	HU-07	OpenAPI	Contrato API	Comparar contrato/rutas	Pendiente

Las rutas y clases de la matriz deben confirmarse con el repositorio. La trazabilidad completa requiere vínculo con el código real y evidencia de ejecución.

8. CONCLUSIONES Y LECCIONES APRENDIDAS

El análisis reconoce el valor de la arquitectura modular y la persistencia relacional centralizada, pero también los riesgos del acoplamiento entre lógica y datos. El documento base señala oportunidades de mejora en seguridad de endpoints, autorización por roles, paginación, importación masiva, consistencia de respuestas, observabilidad y concurrencia.

La estandarización mediante requisitos identificados, diagramas editables, ADR, contratos API y matrices de trazabilidad mejora la comunicación entre análisis, desarrollo y QA. API-First permite acordar el contrato antes de consumirlo desde Angular; BDD expresa comportamientos verificables en lenguaje compartido. Ambos enfoques exigen sincronización con la implementación.

C4 comunica el sistema desde contexto hasta componentes y clases críticas. Microservicios y persistencia políglota pueden aportar ventajas, pero deben adoptarse gradualmente según necesidades medibles y capacidad operativa. Las mejoras no deben declararse completadas sin código, pruebas y evidencias.

ANEXOS Y EVIDENCIAS PENDIENTES

Antes de entregar, adjuntar o completar: (1) UML y C4 editables/exportados; (2) ADR-001 y ADR-002 en Markdown; (3) backlog con historias INVEST/3C y enlace al tablero; (4) OpenAPI 3.0.3 alineado a rutas reales; (5) colección Postman con URLs completas; (6) archivos .feature y resultados de pruebas; (7) prototipo Figma navegable; (8) reporte UX/WCAG; (9) plan SQA y reporte de defectos; (10) matriz de trazabilidad validada.

Nota de alcance

Informe elaborado a partir del PDF del proyecto compartido y de la estructura solicitada. Las integraciones de PostgreSQL, MongoDB, Redis y Neo4j, así como los patrones y escenarios propuestos, no se presentan como implementaciones confirmadas. La conformidad normativa requiere revisión y aprobación.